Jakarta Persistence Applied 온라인 부록

Oracle SH Persistence

권진

Version 1.0: Jakarta Persistence 3.2 판

Table of Contents

Oracle SH Persistence 1
oracle-sample-schemas-persistence-sh 2
SH (Sales History) 4
Attribute Mappings 7
Exercises 29

Oracle SH Persistence

이 장에서는 Oracle 의 예제 스키마 가운데 하나인 SH(Sales History)를 매핑한다. 한 회사의 판매 내역을 분석하기 위해 설계한 스키마다.

지금까지 매핑한 데이터베이스는 업무를 처리하는 데이터베이스였다. SH 는 쌓인 판매 내역을 읽고 집계하는 데이터 웨어하우스의 형태다. 그래서 이 장에서 다루는 문제도 다르다. 기본 키가 없는 테이블에 식별자를 정해야 하고, 뷰와 구체화된 뷰를 읽기 전용 엔티티로 매핑해야 한다.

모듈의 구성과 매핑의 규칙, Oracle 의 형, 테스트, 데이터베이스를 시작하는 방법과 구체화된 뷰는 [oracle-sample-schemas-persistence] 장에서 보았다. 먼저 SH 를 매핑한 `oracle-sample-schemas-persistence-sh` 모듈에만 있는 클래스를 살펴본다. 다음으로 SH 의 테이블과 그 관계, 기본 키가 없는 테이블과 뷰에 식별자를 정하는 방법과 파티션 테이블을 보고, 컬럼을 속성에 어떻게 매핑했는지 주제마다 살펴본다. 마지막으로 연습문제를 모았다.

oracle-sample-schemas-persistence-sh

`oracle-sample-schemas-persistence-sh` 는 SH 를 Jakarta Persistence 로 매핑한 모듈이다. SH 가 어떤 스키마인지는 다음 절인 SH (Sales History) 에서 살펴보고, 이 절에서는 모듈이 어떻게 구성되어 있는지를 먼저 본다.

모듈의 구조와 `module-info.java`, 테스트의 구성은 세 모듈이 같으므로 [oracle-sample-schemas-persistence] 장에서 CO 의 모듈로 보았다. 이 절에서는 SH 의 모듈에만 있는 클래스를 본다.

메인 코드

`sh.mapped` 에는 테이블 9 개와 뷰 1 개, 구체화된 뷰 2 개마다 매핑된 슈퍼클래스가 하나씩, 모두 12 개 있다. `MappedChannel` 부터 `MappedFweekPscatSalesMv` 까지, 이름은 `Mapped` 뒤에 엔티티의 이름을 붙인 것이다. 매핑된 슈퍼클래스 말고도 아래의 클래스가 있다.

Table 1. sh.mapped 패키지의 다른 클래스

클래스	하는 일
`SalesId`, `FweekPscatSalesMvId`	`SALES` 와 `FWEEK_PSCAT_SALES_MV` 의 식별자. `@EmbeddedId` 로 쓰는 임베더블 클래스다.
`CostId`, `ProfitId`	`COSTS` 와 `PROFITS` 의 식별자. `@IdClass` 로 지정하는 보통의 클래스다.
`CountrySection`	`COUNTRIES` 의 지역, 하위 지역, 전체처럼 이름과 그 번호를 함께 담는 임베더블 클래스. 아직 어느 엔티티도 이것을 임베드하지 않는다.
`PromotionNamedId` 와 `PromotionCategoryId`, `PromotionSubcategoryId`, `PromotionTotalId`	`PROMOTIONS` 의 분류처럼 이름과 그 번호를 함께 담는 매핑된 슈퍼클래스와, 그것을 상속한 세 임베더블 클래스. 역시 아직 어느 엔티티도 임베드하지 않는다.

테스트 코드

테스트 코드의 `sh` 패키지에는 매핑된 슈퍼클래스를 상속한 엔티티가 12 개 있다. SH 에는 CO 와 달리 키가 없는 뷰를 보통의 클래스로 매핑한 것이 없다. 키가 없는 테이블과 뷰에도 식별자를 정해(식별자를 정하는 방법) 모두 엔티티로 매핑하기 때문이다.

Table 2. SH 의 엔티티

테이블 또는 뷰	엔티티
CHANNELS	Channel
COUNTRIES	Country
CUSTOMERS	Customer
SUPPLEMENTARY_DEMOGRAPHICS	SupplementaryDemographics
PRODUCTS	Product
PROMOTIONS	Promotion
TIMES	Time
SALES	Sales
COSTS	Cost
PROFITS	Profit
CAL_MONTH_SALES_MV	CalMonthSalesMv
FWEEK_PSCAT_SALES_MV	FweekPscatSalesMv

[oracle-sample-schemas-persistence-tests] 에서 본 테스트 클래스는 12 개 엔티티 모두에 있고, 퍼시스터만 테이블의 엔티티 9 개에 있다.

그 밖의 테스트는 아래와 같다.

- SalesUtils, FweekPscatSalesMvUtils, TimeUtils, CalMonthSalesMvUtils 는 엔티티의 식별자나 속성에서 값을 꺼내거나 바꾸어 넣는 메서드를 모은 클래스다. 엔티티에 두면 그 메서드가 엔티티의 속성처럼 보이므로 따로 두었다. 클래스마다 *_Test, *_PersistenceTest, *_PersistenceIT 가 있다.
- Time_AttributeConverters_Test 는 영속성 유닛이 TIMES 의 문자열 컬럼을 위해 등록한 jinahya-persistence-more 의 변환기를 검사한다(문자열).
- CountrySection_Test 와 PromotionCategoryId_Test, PromotionSubcategoryId_Test, PromotionTotalId_Test 는 임베더블 클래스를 검사하고, PromotionNamedId_Test 는 뒤의 셋이 함께 상속하는 추상 클래스다.

테스트 코드의 sh.mapped 패키지에는 식별자 클래스 넷의 테스트와 메인 코드의 sh.mapped 에 있는 클래스의 랜덤마이저가 있다.

SH (Sales History)

SH 는 Oracle 이 배포하는 예제 스키마로, 한 회사의 판매 거래와 보고서를 모델링한다.[1] 이 장이 쓰는 것은 스키마의 21.1 판이다.

스키마에는 테이블 아홉 개와 뷰 하나, 구체화된 뷰 두 개가 있다.[2] 구체화된 뷰가 무엇이고 어떻게 매핑하는지는 [oracle-materialized-views] 에서 보았다.

사실 테이블과 차원 테이블

SH 는 데이터 웨어하우스에서 흔히 쓰는 스타 스키마(star schema)의 형태다. 가운데에 판매 한 건 한 건을 기록하는 **사실 테이블**(事實, fact table) `SALES` 가 있고, 그 둘레에 판매를 설명하는 차원 테이블(次元, dimension table)이 있다. 누가 샀는지는 `CUSTOMERS`, 무엇을 샀는지는 `PRODUCTS`, 언제 샀는지는 `TIMES`, 어떤 경로로 샀는지는 `CHANNELS`, 어떤 판촉으로 샀는지는 `PROMOTIONS` 가 담는다. 스키마를 만드는 스크립트도 테이블마다 붙인 주석에서 `SALES` 를 사실 테이블로, 나머지를 차원 테이블로 부른다.

`COSTS` 는 상품의 원가와 단가를 날짜, 판촉, 판매 경로마다 기록하는 또 하나의 사실 테이블이다. `COUNTRIES` 는 `CUSTOMERS` 가 참조하는 나라의 차원이다. 차원이 다른 차원을 거쳐 나뉘어 있으므로, 스크립트는 이것을 눈송이(snowflake)라고 부른다. `SUPPLEMENTARY_DEMOGRAPHICS` 는 고객의 인구 통계 정보를 덧붙인다.

Table 3. SH 의 테이블

묶음	테이블
판매와 원가	`SALES`, `COSTS`, `PRODUCTS`, `TIMES`, `CHANNELS`, `PROMOTIONS`
고객	`CUSTOMERS`, `COUNTRIES`, `SUPPLEMENTARY_DEMOGRAPHICS`

뷰 `PROFITS` 는 `SALES` 와 `COSTS` 를 조인해 판매마다 원가를 붙이고, 두 구체화된 뷰는 `SALES` 의 판매 금액을 합해 둔다.

묶음마다 테이블과 그 사이의 관계를 그림으로 보면 아래와 같다. 그림에는 같은 묶음 안의 관계만 그렸다.

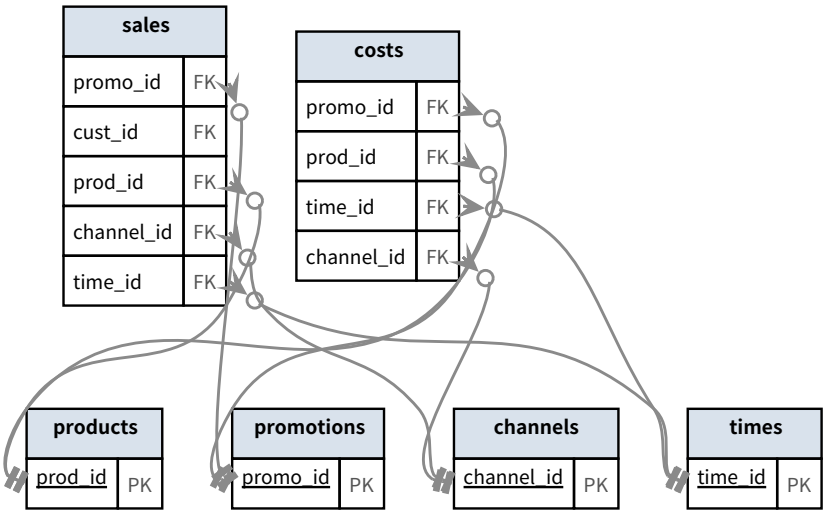


Figure 1. 판매와 원가

Table 4. 판매와 원가 묶음

테이블	담는 것	행
SALES	사실 테이블. 판매 한 건의 수량과 금액. 2019 년부터 2022 년까지	918,843
COSTS	사실 테이블. 상품의 원가와 단가. 2019 년부터 2022 년까지	82,112
PRODUCTS	상품의 차원. 이름과 분류, 정가와 최저가	72
TIMES	시간의 차원. 2019 년부터 2023 년까지 하루에 한 행	1,826
CHANNELS	판매 경로의 차원. 직접, 인터넷, 카탈로그, 협력사	5
PROMOTIONS	판촉의 차원. 이름과 분류, 비용, 시작하고 끝나는 날	503

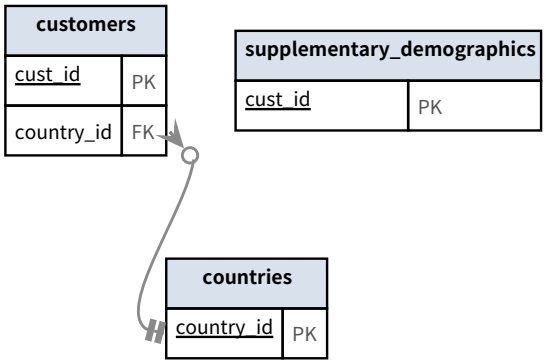


Figure 2. 고객

Table 5. 고객 묶음

테이블	담는 것	행
CUSTOMERS	고객의 차원. 이름과 주소, 소득 수준과 신용 한도	55,500
COUNTRIES	나라의 차원(눈송이). 하위 지역과 지역의 계층	35
SUPPLEMENTARY_DEMOGRAPHICS	고객의 인구 통계 정보. 학력과 직업, 가구의 크기	4,500

데이터

테이블의 행 수는 위의 표에서 보았다. 뷰와 구체화된 뷰에는 아래만큼의 행이 들어 있다.

Table 6. SH 의 뷰와 구체화된 뷰의 행 수

뷰	행	비고
PROFITS	916,039	뷰
CAL_MONTH_SALES_MV	0	구체화된 뷰. 갱신하면 48
FWEEK_PSCAT_SALES_MV	0	구체화된 뷰. 갱신하면 10,512

두 구체화된 뷰가 설치가 끝나도 비어 있는 이유는 [oracle-materialized-views] 에서 보았다.

식별자를 정하는 방법

엔티티에는 식별자가 있어야 한다.[3] 차원 테이블 일곱 개에는 기본 키가 있으므로, 기본 키를 그대로 식별자로 쓴다. 그런데 두 사실 테이블 SALES 와 COSTS 에는 기본 키도 유일 제약도 없다. 차원을 참조하는 컬럼에 유일하지 않은 비트맵 인덱스가 있을 뿐이다. 뷰와 구체화된 뷰에도 키가 없다.

SH (Sales History)

이 프로젝트는 키가 없는 테이블과 뷰마다 데이터를 측정해 식별자로 쓸 컬럼을 정했다. 행 수와 컬럼마다 `NULL` 이 아닌 값의 수, 서로 다른 값의 수를 세어 후보를 찾고, 그 컬럼들로 묶어 두 번 이상 나오는 조합이 없는지 확인한다.

Table 7. SH 의 식별자

테이블 또는 뷰	식별자의 컬럼	근거	매핑
차원 테이블 일곱 개	기본 키	기본 키	`@Id`
`SALES`	`PROD_ID, CUST_ID, TIME_ID, CHANNEL_ID, PROMO_ID`	측정	`@EmbeddedId (SalesId)`
`COSTS`	`PROD_ID, TIME_ID, PROMO_ID, CHANNEL_ID`	측정	`@IdClass (CostId)`
`PROFITS`	`CHANNEL_ID, CUST_ID, PROD_ID, PROMO_ID, TIME_ID`	측정	`@IdClass (ProfitId)`
`CAL_MONTH_SALES_MV`	`CALENDAR_MONTH_DESC`	`GROUP BY`	`@Id`
`FWEEK_PSCAT_SALES_MV`	`WEEK_ENDING_DAY, PROD_SUBCATEGORY, CHANNEL_ID, PROMO_ID`	`GROUP BY`	`@EmbeddedId ( FweekPscatSalesMvId)`

복합 식별자는 두 방법으로 매핑할 수 있다. 이 프로젝트는 두 방법을 모두 보이려고, 테이블 둘과 뷰 둘에 하나씩 나누어 썼다. 테이블에서는 `SALES` 가 `@EmbeddedId` 를, `COSTS` 가 `@IdClass` 를 쓰고, 뷰에서는 `FWEEK_PSCAT_SALES_MV` 가 `@EmbeddedId` 를, `PROFITS` 가 `@IdClass` 를 쓴다.

측정으로 정한 식별자와 `GROUP BY` 로 정한 식별자는 유일성을 보장하는 근거가 다르다. 구체화된 뷰는 정의한 쿼리가 그 컬럼들로 묶으므로, 어떤 데이터가 들어와도 같은 조합이 두 번 나오지 않는다. **측정으로 정한 식별자는 지금 들어 있는 데이터에서 그렇다는 것일 뿐, 데이터베이스가 보장하지 않는다.**

같은 고객이 같은 날 같은 경로로 같은 판촉의 같은 상품을 두 번 사는 일은 판매 기록에서 얼마든지 있을 수 있고, `SALES` 는 그런 행을 막지 않는다. 그런 두 행이 생기면, 영속성 컨텍스트에는 한 식별자에 한 인스턴스만 있으므로 두 행을 읽는 쿼리는 같은 인스턴스를 두 번 돌려준다.[4] 둘째 행의 `QUANTITY_SOLD` 와 `AMOUNT_SOLD` 는 아무 경고 없이 결과에서 빠진다. 기본 키가 없는 테이블에 식별자를 정할 때에는 이 위험을 알고 정해야 한다.

파티션

두 사실 테이블 `SALES` 와 `COSTS` 는 `TIME_ID` 의 범위로 나눈 파티션 테이블이다. `SALES` 는 2018 년까지, 2019 년의 두 반기, 2020 년부터 2022 년까지의 분기마다 파티션이 하나씩, 모두 15 개이고, `COSTS` 는 20 개다.

SALES 의 파티션 정의 가운데 처음 두 개

```
PARTITION BY RANGE (time_id)
(
   PARTITION SALES_2018 VALUES LESS THAN
      (TO_DATE('2019-01-01','YYYY-MM-DD','NLS_DATE_LANGUAGE = American')),
   PARTITION SALES_H1_2019 VALUES LESS THAN
      (TO_DATE('2019-07-01','YYYY-MM-DD','NLS_DATE_LANGUAGE = American')),
```

옵티마이저는 쿼리의 `FROM` 과 `WHERE` 를 보고 필요 없는 파티션을 읽지 않는다.[5] `TIME_ID` 의 범위로 판매를 선택하는 쿼리는 그 범위의 파티션만 읽는다.

매핑에는 파티션에 관한 정보가 없다. `@Table` 에는 테이블의 이름만 기재하고, 구현체가 생성하는 SQL 도 파티션을 가리키지 않는다. 파티션을 나누는 일도, 쿼리에 맞는 파티션을 선택하는 일도 데이터베이스가 한다. 엔티티의 쪽에서 할 일은 `TIME_ID` 로 범위를 거는 쿼리를 쓰는 것이다.

Attribute Mappings

SH 의 테이블 9 개에서 매핑된 슈퍼클래스가 컬럼을 매핑한 속성은 모두 136 개이고, 컬럼마다 속성이 꼭 하나씩 있다. Oracle 의 형을 어떤 자바의 형으로 매핑하는지는 [oracle-types] 에서 보았다. 이 절은 먼저 그 규칙을 SH 의 어느 컬럼에 적용했는지 요약하고, 그다음 SH 가 매핑에 더하는 것을 주제마다 살펴본다. 테이블을 하나씩 살펴보지는 않는다.

요약

- VARCHAR2 와 CHAR 는 String 이다(문자열). TIMES 의 요일, 달, 연월도 변환기 없이 String 으로 둔다(문자열과 수로 기재한 날짜).
- 식별자와 개수는 대부분 정밀도가 없는 NUMBER 이고, Long 이다(수).
- 날짜를 담는 컬럼은 모두 DATE 이고, LocalDateTime 이다(날짜와 시각).
- 차원 테이블은 계층의 단계마다 이름과 번호를 두 컬럼에 담고, 매핑된 슈퍼클래스는 두 컬럼을 두 속성으로 매핑한다(이름과 번호의 쌍).
- 기본 키가 있는 차원 테이블 일곱은 기본 키를 @Id 로 매핑한다. 기본 키가 없는 SALES 와 COSTS, 뷰와 구체화된 뷰는 식별자를 정하는 방법 에서 정한 컬럼을 @EmbeddedId 나 @IdClass 로 매핑한다(기본 키가 없는 사실 테이블, 뷰 PROFITS, 구체화된 뷰).
- 외래 키 컬럼은 두 번 매핑한다(외래 키와 관계). SALES 는 식별자 전체를 @MapsId 로 연관 관계에서 가져오고(SALES 와 @EmbeddedId), COSTS 는 식별자인 속성이 컬럼을 쓴다(COSTS 와 @IdClass).
- 뷰와 구체화된 뷰까지 세면 속성은 153 개다. 집계한 금액은 BigDecimal 이다(수).

문자열

SH 는 VARCHAR2 의 길이를 바이트로 센다. @Size 를 통과한 값도 데이터베이스가 거부할 수 있다는 것은 [oracle-types] 에서 보았다. CHAR 인 컬럼은 COUNTRIES.COUNTRY_ISO_CODE, CUSTOMERS.CUST_GENDER, TIMES 의 분기를 나타내는 두 컬럼처럼 길이가 정해진 코드다.

TIMES 에는 DayOfWeek, Month, YearMonth 로 읽을 수 있는 문자열 컬럼이 있지만, 매핑된 슈퍼클래스는 이 컬럼들도 String 으로 둔다. 그 변환기는 문자열과 수로 기재한 날짜 절에서, 같은 연월을 식별자로 담는 컬럼은 CAL_MONTH_SALES_MV 절에서 본다.

NULL 을 허용하는 컬럼은 @Basic(optional = true) 와 nullable = true 로 매핑하고, @NotNull 을 붙이지 않는다. CUSTOMERS 의 혼인 여부 CUST_MARITAL_STATUS 가 그렇다.

MappedCustomer 의 custMaritalStatus

```
@Size(max = SIZE_MAX_CUST_MARITAL_STATUS)
@Basic(optional = true)
@Column(name = COLUMN_NAME_CUST_MARITAL_STATUS,
        nullable = true,
        insertable = true,
        updatable = true,
        length = COLUMN_LENGTH_CUST_MARITAL_STATUS,
        comment = "customer marital status; low cardinality attribute"
)
private String custMaritalStatus;
```

설치한 데이터에서는 고객 55,500 명 가운데 17,506 명, 셋 가운데 하나꼴로 혼인 여부가 NULL 이다. 이 컬럼으로 묶거나 조건을 거는 쿼리는 NULL 인 행을 따로 다루어야 한다.

수

SH 는 식별자와 그것을 참조하는 컬럼, 개수를 대부분 정밀도가 없는 NUMBER 로 선언했다. [oracle-types] 에서 본 대로, 이런 컬럼은 Long 으로 매핑한다.

SUPPLEMENTARY_DEMOGRAPHICS 에 있는 NUMBER(10) 컬럼 여덟 개는 아홉 자리를 넘으므로 Long 이다. 가격과 원가, 판매 금액은 소수(小數, decimal fraction) 자리가 있으므로 BigDecimal 이다.

뷰와 구체화된 뷰가 집계한 DOLLARS 와 TOTAL_COST 는 금액이므로 BigDecimal 이다.

식별자

차원 테이블의 식별자는 @Id 로 매핑하고, 그 값은 애플리케이션이 정한다. CHANNELS 의 식별자는 2, 3, 4, 5, 9 처럼 번호가 이어지지도 않는다. 정밀도가 없는 NUMBER 인 CHANNELS.CHANNEL_ID 는 Long 이다.

MappedChannel 의 channelId

```
@NotNull
@Id
@Column(name = COLUMN_NAME_CHANNEL_ID, nullable = false, insertable = true, updatable = false,
        comment = "primary key column")
private Long channelId;
```

PRODUCTS.PROD_ID 는 NUMBER(6) 이다. 여섯 자리까지의 정수이므로 Integer 로 매핑하고, 담을 수 있는 범위를 @Min 과 @Max 로 기재한다. PROMOTIONS.PROMO_ID 도 NUMBER(6) 이므로 Integer 다.

MappedProduct 의 prodId

```
@NotNull
@Id
@Max(ATTRIBUTE_MAX_PROD_ID)
@Min(ATTRIBUTE_MIN_PROD_ID)
@Column(name = COLUMN_NAME_PROD_ID, nullable = false, insertable = true, updatable = false,
        comment = "primary key")
private Integer prodId;
```

외래 키 컬럼은 참조하는 키의 형을 따른다([oracle-types]). SALES.CHANNEL_ID 는 NUMBER(1) 이지만, 참조하는 CHANNELS.CHANNEL_ID 가 NUMBER 이므로 Long 이다. 거꾸로 COSTS.PROD_ID 와 COSTS.PROMO_ID 는 NUMBER 이지만, 참조하는 PRODUCTS.PROD_ID 와 PROMOTIONS.PROMO_ID 가 NUMBER(6) 이므로 Integer 다.

금액

PRODUCTS 의 정가 PROD_LIST_PRICE 와 최저가 PROD_MIN_PRICE 는 NUMBER(8,2) 다. 금액이므로 BigDecimal 로 매핑하고, 컬럼이 담을 수 있는 가장 작은 값과 가장 큰 값을 @DecimalMin 과 @DecimalMax 로 기재한다.

MappedProduct 의 prodListPrice

```
@NotNull
@DecimalMax(value = ATTRIBUTE_DECIMAL_MAX_PROD_LIST_PRICE, inclusive = true) ❶
@DecimalMin(value = ATTRIBUTE_DECIMAL_MIN_PROD_LIST_PRICE, inclusive = true)
@Basic(optional = false)
@Column(name = COLUMN_NAME_PROD_LIST_PRICE,
        nullable = false,
        insertable = true,
        updatable = true,
        precision = COLUMN_PRECISION_PROD_LIST_PRICE,                    ❷
        scale = COLUMN_SCALE_PROD_LIST_PRICE,
        comment = "product list price"
)
private BigDecimal prodListPrice;
```

❶ 값은 +999999.99 다. 정수(整數, integer) 부분이 여섯 자리, 소수(小數, decimal fraction) 부분이 두 자리다.

❷ 테이블을 만들 때 쓰는 정보다. NUMBER(8,2) 의 8 과 2 다.[6]

설치한 데이터의 정가는 6.99 부터 1,299.99 까지다. 최저가가 정가보다 높은 상품은 없지만, 그것을 지키는 제약도 없다.

날짜와 시각

TIME_ID 와 판촉의 시작일과 종료일처럼 날짜를 담는 컬럼은 모두 DATE 이므로, 모두 LocalDateTime 으로 매핑한다([oracle-types]).

TIMES 의 식별자 TIME_ID 는 하루를 가리킨다. 설치한 SH 에는 2019 년 1 월 1 일부터 2023 년 12 월 31 일까지 1,826 일이 들어 있고, 모든 행의 시각은 0 시 0 분 0 초다.

MappedTime 의 timeId

```
@NotNull
@Id
@Column(name = COLUMN_NAME_TIME_ID, nullable = false, insertable = true, updatable = false,
        comment = "primary key; day date, finest granularity, CORRECT ORDER")
private LocalDateTime timeId;
```

TIMES 의 시각은 모두 0 시이므로, 날짜만 필요하면 toLocalDate() 로 얻는다.

문자열과 수로 기재한 날짜

TIMES 는 하루에 한 행이 있고, 그날이 무슨 요일인지, 어느 주, 어느 달, 어느 분기, 어느 해에 속하는지를 미리 계산해 담는다. 달력의 계층과 회계 연도의 계층을 함께 담으므로 컬럼이 38 개나 된다. 그 많은 컬럼은 자바에 더 알맞은 형이 있는 값을 문자열이나 수로 담는다.

2021 년의 첫 사흘을 보면 아래와 같다.

2021 년의 첫 사흘

```
TIME_ID    DAY_NAME  CALENDAR_MONTH_DESC CALENDAR_MONTH_NAME CALENDAR_QUARTER_DESC
---------- --------- ------------------- ------------------- ---------------------
2021-01-01 Friday    2021-01             January             2021-01
2021-01-02 Saturday  2021-01             January             2021-01
2021-01-03 Sunday    2021-01             January             2021-01
```

DAY_NAME 의 Friday 는 java.time.DayOfWeek 의 FRIDAY 이고, CALENDAR_MONTH_NAME 의 January 는 java.time.Month 의 JANUARY 이며, CALENDAR_MONTH_DESC 의 2021-01 은 java.time.YearMonth 의 2021-01 이다.

스크립트의 주석은 분기를 1998-Q1 처럼 기재한다고 하지만, 설치한 데이터의 분기는 2021-01 처럼 달과 형태가 같다. 뒤의 두 자리는 달이 아니라 분기의 번호다.

MappedTime 은 이 컬럼들을 모두 String 으로 매핑한다.

MappedTime 의 dayName

```
@Size(max = SIZE_MAX_DAY_NAME)
@NotNull
@Basic(optional = false)
@Column(name = COLUMN_NAME_DAY_NAME,
        nullable = false,
        insertable = true,
        updatable = true,
        length = COLUMN_LENGTH_DAY_NAME,
        comment = "Monday to Sunday, repeating"
)
private String dayName;
```

변환기

속성을 DayOfWeek 나 Month, YearMonth 로 두고 변환기로 문자열과 바꿀 수도 있다. 그러려고 쓸 수 있는 변환기가 jinahya-persistence-more 에 셋 있다.

Table 8. TIMES 의 변환기

변환기	속성의 형	컬럼
__TemporalAccessorTextStringAttributeConverters.OfDayOfWeek	DayOfWeek	DAY_NAME
__TemporalAccessorTextStringAttributeConverters.OfMonth	Month	CALENDAR_MONTH_NAME, FISCAL_MONTH_NAME
__TemporalAccessorStringAttributeConverters.OfYearMonth	YearMonth	CALENDAR_MONTH_DESC, FISCAL_MONTH_DESC

그런데 세 변환기 모두 아직 어느 속성에도 적용하지 않았다. @Convert 로 기재한 속성이 없고, 테스트의 영속성 유닛이 세 변환기를 관리 클래스로 기재해 두었을 뿐이다. 속성에 적용하는 일은 연습문제로 남겨 두었다.

TimeUtils

그 대신 이 프로젝트는 문자열로 읽은 값을 필요할 때 바꾼다. `TimeUtils` 는 `Time` 의 속성을 읽어 자바의 형으로 바꾸거나, 자바의 값을 문자열로 바꾸어 속성에 넣는 정적 메서드를 모아 둔다. 바꾸는 일은 위의 세 변환기에 맡긴다.

TimeUtils 의 getDayNameAsDayOfWeek 와 setDayNameFromDayOfWeek

```
// 영속성 유닛이 이 컬럼들을 위해 기재한 변환기; 저마다 불변이므로 인스턴스 하나로 충분하다
private static final __TemporalAccessorTextStringAttributeConverters.OfDayOfWeek DAY_NAME =
        new __TemporalAccessorTextStringAttributeConverters.OfDayOfWeek();

public static DayOfWeek getDayNameAsDayOfWeek(final Time time) {
    return getDayNameAsMapped(time, DAY_NAME::convertToEntityAttribute);
}

public static void setDayNameFromDayOfWeek(final Time time, final DayOfWeek dayOfWeek) {
    setDayNameFromMapped(time, dayOfWeek, DAY_NAME::convertToDatabaseColumn);
}
```

변환기와 같은 일을 엔티티 밖에서 한다. 이 메서드들을 `Time` 에 두지 않은 것은 `getDayNameAsDayOfWeek()` 같은 메서드가 엔티티에 있으면 자바빈의 속성처럼 보이기 때문이다. `Time` 의 getter 와 setter 는 매핑한 속성의 것뿐이다.

변환기를 적용하려면

`dayName` 을 `DayOfWeek` 로 바꾸고 `@Convert(converter = __TemporalAccessorTextStringAttributeConverters.OfDayOfWeek.class)` 를 붙이면 변환기가 적용된다. 바뀌는 것은 그 속성 하나로 끝나지 않는다. 속성의 형이 바뀌므로 getter 와 setter, 정적 메타모델의 형, 그 속성을 문자열과 비교하던 쿼리가 모두 바뀐다. 변환한 속성을 쿼리에서 리터럴이나 매개변수와 비교하면, 그 값도 변환기를 거쳐야 한다.[7] 이 프로젝트는 그 일을 연습으로 남겨 두었다.

`CALENDAR_QUARTER_DESC` 에는 `OfYearMonth` 를 적용할 수 없다. `2021-04` 를 `YearMonth` 로 읽으면 4 월이 되지만, 이 컬럼의 `2021-04` 는 4 분기다. **형태가 같은 문자열이라도 뜻이 다르면 같은 변환기를 쓸 수 없다.**

CAL_MONTH_SALES_MV 절에서는 같은 `CALENDAR_MONTH_DESC` 가 식별자가 되어, 변환기를 적용할 수 없는 경우를 본다.

이름과 번호의 쌍

판매 경로의 차원 테이블 `CHANNELS` 에는 행이 다섯 개 있다. 모두 보면 아래와 같다.

CHANNELS 의 행

```
CHANNEL_ID CHANNEL_DESC         CHANNEL_CLASS        CHANNEL_CLASS_ID
---------- -------------------- -------------------- ----------------
         2 Partners             Others                             14
         3 Direct Sales         Direct                             12
         4 Internet             Indirect                           13
         5 Catalog              Indirect                           13
         9 Tele Sales           Direct                             12
```

`CHANNEL_CLASS` 와 `CHANNEL_CLASS_ID` 는 경로가 속한 분류를, 쿼리에서 뺀 `CHANNEL_TOTAL` 과 `CHANNEL_TOTAL_ID` 는 모든 경로를 합한 단계를 가리킨다. 뒤의 두 컬럼은 모든 행이 `Channel total` 과 1 이다. 경로, 분류, 전체로 이어지는 계층이다. 집계할 때 분류의 이름 대신 번호로 묶을 수 있도록, 단계마다 이름과 번호를 함께 둔다.

이 쌍은 SH 의 차원 테이블마다 되풀이된다.

Table 9. 계층의 단계마다 이름과 번호를 담는 컬럼

테이블	단계	컬럼
CHANNELS	분류, 전체	CHANNEL_CLASS, CHANNEL_TOTAL 과 그 _ID
COUNTRIES	하위 지역, 지역, 전체	COUNTRY_SUBREGION, COUNTRY_REGION, COUNTRY_TOTAL 과 그 _ID
CUSTOMERS	도시, 주, 전체	CUST_CITY, CUST_STATE_PROVINCE, CUST_TOTAL 과 그 _ID
PRODUCTS	하위 분류, 분류, 전체	PROD_SUBCATEGORY, PROD_CATEGORY, PROD_TOTAL 과 그 _ID
PROMOTIONS	하위 분류, 분류, 전체	PROMO_SUBCATEGORY, PROMO_CATEGORY, PROMO_TOTAL 과 그 _ID

CUSTOMERS 는 도시, 주, 나라, 전체로 이어지는 계층인데, 나라의 단계만 COUNTRIES 로 나뉘어 있다. 이름의 컬럼은 VARCHAR2 이고, 번호의 컬럼은 정밀도가 없는 NUMBER 이므로 Long 이다.

두 속성으로 매핑한다

매핑된 슈퍼클래스는 쌍의 두 컬럼을 그냥 두 속성으로 매핑한다.

MappedChannel 의 channelClass 와 channelClassId

```
@Size(max = SIZE_MAX_CHANNEL_CLASS)
@NotNull
@Basic(optional = false)
@Column(name = COLUMN_NAME_CHANNEL_CLASS,
        nullable = false,
        insertable = true,
        updatable = true,
        length = COLUMN_LENGTH_CHANNEL_CLASS,
        comment = "e.g. direct, indirect"
)
private String channelClass;

@NotNull
@Basic(optional = false)
@Column(name = COLUMN_NAME_CHANNEL_CLASS_ID,
        nullable = false,
        insertable = true,
        updatable = true
)
private Long channelClassId;
```

MappedCountry 도 같다.

MappedCountry 의 countryRegion 과 countryRegionId

```
@Size(max = SIZE_MAX_COUNTRY_REGION)
@NotNull
@Column(name = COLUMN_NAME_COUNTRY_REGION, nullable = false,
        length = COLUMN_LENGTH_COUNTRY_REGION,
        comment = "e.g. Europe, Asia")
private String countryRegion;

@NotNull
@Column(name = COLUMN_NAME_COUNTRY_REGION_ID, nullable = false)
private Long countryRegionId;
```

MappedChannel 과 달리 @Basic 을 붙이지 않고, @Column 에도 insertable 과 updatable 을 기재하지 않았다. 둘 다 기본값이 true 이므로 뜻은 같다.[8]

쌍이 어긋난 행

이름과 번호를 따로 기재하면 둘이 어긋날 수 있다. 설치한 데이터에도 그런 행이 있다.

지역이 Americas 인 나라

```
COUNTRY_NAME             COUNTRY_SUBREGION COUNTRY_SUBREGION_ID COUNTRY_REGION_ID
------------------------ ----------------- -------------------- -----------------
Canada                   Northern America                52797             52801
Argentina                Southern America                52798             52801
Brazil                   Southern America                52798             52801
United States of America Northern America                52797             52801
Mexico                   Northern America                52797             52801
Chile                    Southern America                52798             52798
```

지역의 이름은 모두 Americas 인데, 칠레의 지역 번호만 52801 이 아니라 52798 이다. 52798 은 칠레가 속한 하위 지역 Southern America 의 번호다. 지역을 이름으로 묶으면 아메리카의 나라는 여섯이고, 번호로 묶으면 다섯과 하나로 갈라진다. 테이블에는 이름과 번호가 맞는지 확인하는 제약이 없다.

상품의 분류에서도 그렇다. 분류마다 하위 분류와 상품을 세면 아래와 같다.

분류마다 하위 분류와 상품의 수

```
PROD_CATEGORY        PROD_CATEGORY_ID SUBCATEGORIES   PRODUCTS
-------------------- ---------------- -------------- ----------
Tennis                            201              4         13
Golf                              202              2          2
Baseball                          203              6         21
Soccer / Football                 204              4         10
Cricket                           205              5         26
```

그런데 하위 분류의 이름은 19 가지이고 번호는 21 가지다. `Tennis` 의 하위 분류를 보면 그 형태가 드러난다.

Tennis 의 하위 분류

```
PROD_SUBCATEGORY     PROD_SUBCATEGORY_ID   PRODUCTS
-------------------- -------------------- ----------
Tennis Racquet                      2011          1
Tennis Racquet                      2012          2
Tennis Balls                        2013          1
Tennis Racquet Grip                 2013          1
Tennis Balls                        2014          3
Tennis Racquet Grip                 2014          1
Tennis Strings                      2014          4
```

같은 이름에 번호가 둘이고, 같은 번호에 이름이 여럿이다. 하위 분류로 묶을 때 이름으로 묶는지 번호로 묶는지에 따라 결과가 다르다. FWEEK_PSCAT_SALES_MV 절의 구체화된 뷰는 이름으로 묶는다.

CountrySection

이름과 번호의 쌍을 하나의 값으로 다루려면 임베더블 클래스로 묶을 수 있다. 이 프로젝트에는 그러려고 만든 `CountrySection` 이 있다. `COUNTRIES` 의 하위 지역, 지역, 전체 가운데 한 단계를 이름과 번호로 담는다. 그러나 아직 이 클래스를 임베드한 엔티티는 없고, `MappedCountry` 는 쌍을 두 속성으로 매핑한다.

CountrySection

```java
@Embeddable
public class CountrySection {

    @NotNull
    @Basic(optional = false, fetch = FetchType.EAGER)
    // @Column 이 없다: 임베드하는 엔티티가 단계마다 두 컬럼 -- 이름, 길이, NULL 허용 여부 -- 를
    // @AttributeOverride 로 정의하고, 그것이 여기 선언한 @Column 을 대체한다
    private String name;

    @NotNull
    @Basic(optional = false, fetch = FetchType.EAGER)
    private Long id;
}
```

임베더블 클래스의 인스턴스는 자기만의 식별자가 없고, 그것을 가진 엔티티의 상태의 일부로만 존재한다.[9] `CountrySection` 은 컬럼의 이름을 정하지 않는다. 한 엔티티가 같은 임베더블을 세 번 임베드하면 단계마다 컬럼이 달라야 하기 때문이다. 임베드하는 쪽이 `@AttributeOverride` 로 단계마다 `name` 과 `id` 의 컬럼을 정한다.

`equals` 와 `hashCode` 는 이름과 번호를 모두 쓴다. 위의 칠레처럼 이름은 같고 번호가 다른 두 단계는 서로 다른 값이다.

임베드한 엔티티는 없지만, 영속성 유닛은 이 클래스를 관리 클래스로 기재해 둔다.

단계마다 하나의 클래스

PROMOTIONS 에는 같은 일을 다른 방법으로 하는 클래스가 넷 있다. PromotionNamedId 가 이름과 번호를 담고, 그것을 상속한 PromotionSubcategoryId, PromotionCategoryId, PromotionTotalId 가 단계마다 컬럼의 이름을 정한다.

PromotionNamedId

```
@MappedSuperclass
public abstract class PromotionNamedId<N extends Number> {

    private String name;

    private N id;
}
```

PromotionNamedId 는 임베더블이 아니라 매핑된 슈퍼클래스다. 번호의 형은 형 변수 N 으로 남겨 둔다. 컬럼에 관한 정보는 하나도 없다.

PromotionCategoryId

```
@AttributeOverride(name = PromotionNamedId.ATTRIBUTE_NAME_NAME,              ❶
                   column = @Column(name = MappedPromotion.COLUMN_NAME_PROMO_CATEGORY,
                                    nullable = false,
                                    insertable = true,
                                    updatable = true,
                                    length = MappedPromotion.COLUMN_LENGTH_PROMO_CATEGORY,
                                    comment = "promotion category"))
@AttributeOverride(name = PromotionNamedId.ATTRIBUTE_NAME_ID,
                   column = @Column(name = MappedPromotion.COLUMN_NAME_PROMO_CATEGORY_ID,
                                    nullable = false,
                                    insertable = true,
                                    updatable = true))
@Embeddable
public class PromotionCategoryId
        extends PromotionNamedId<Long> {                                     ❷

    @Size(min = MappedPromotion.SIZE_MIN_PROMO_CATEGORY,
          max = MappedPromotion.SIZE_MAX_PROMO_CATEGORY)
    @NotNull
    public String getPromoCategory() {                                       ❸
        return getName();
    }

    @NotNull
    public Long getPromoCategoryId() {
        return getId();
    }
}
```

❶ 상속한 두 속성의 컬럼을 클래스에서 정한다. 매핑된 슈퍼클래스가 정한 매핑은 하위 클래스에서 @AttributeOverride 로 바꿀 수 있다.[10]

❷ 번호의 형을 Long 으로 정한다.

❸ 상속한 name 과 id 를 컬럼의 이름을 따른 이름으로 읽는다. 검증 제약은 필드가 슈퍼클래스에 있으므로 이 getter 에 붙였다.

두 방법의 차이는 컬럼의 이름을 어디서 정하는지다. `CountrySection` 을 임베드하는 엔티티는 단계마다 `@AttributeOverride` 를 둘씩 기재해야 한다. `PromotionCategoryId` 는 클래스가 이미 컬럼을 정했으므로, 임베드하는 엔티티는 `@Embedded` 하나만 기재하면 된다. 그 대신 단계마다 클래스가 하나씩 있어야 한다.

`CountrySection` 처럼 이 클래스들도 아직 임베드한 엔티티가 없다. `MappedPromotion` 은 쌍을 두 속성으로 매핑하고, 영속성 유닛도 네 클래스를 기재하지 않는다. `Promotion` 의 세 쌍을 이 임베더블로 바꾸어 보는 것은 독자에게 남긴다.

따옴표로 감싼 이름

`SUPPLEMENTARY_DEMOGRAPHICS` 는 고객이 즐기는 운동을 `NUMBER(10)` 컬럼마다 `1` 과 `0` 으로 담는데, 그 가운데 하나는 이름이 `UNKNOWN` 이다. 이 이름은 SQL 에서 예약어로 쓰이므로, 이 프로젝트가 테스트에 쓰는 H2 는 따옴표가 없는 `UNKNOWN` 을 컬럼의 이름으로 받아들이지 않는다. 그래서 이 속성만 컬럼의 이름을 따옴표로 감싼다.

MappedSupplementaryDemographics 의 unknown

```
@Max(ATTRIBUTE_MAX_UNKNOWN)
@Min(ATTRIBUTE_MIN_UNKNOWN)
@Basic(optional = true)
// 따옴표로 감싼다: UNKNOWN 은 예약된 식별자라 따옴표가 없으면 H2 가 생성한 DDL 을 거부한다. 그리고
// 따옴표로 감싼 이름은 Oracle 이 가진 대문자 이름이므로, 설치한 스키마에서도 매핑은 그대로 맞다
@Column(name = '"' + COLUMN_NAME_UNKNOWN + '"', ❶
        nullable = true,
        insertable = true,
        updatable = true
)
private Long unknown;
```

❶ 컬럼의 이름은 `"UNKNOWN"` 이다. 큰따옴표는 이름의 일부가 아니라, 이름을 그대로 쓰라는 표시다.

Oracle 은 따옴표 없이 만든 이름을 대문자로 바꾸어 저장한다. 데이터 사전에서 이 컬럼의 이름은 `UNKNOWN` 이다. 따옴표로 감싼 이름은 대소문자를 바꾸지 않으므로, 대문자로 기재한 `"UNKNOWN"` 은 Oracle 에서도 같은 컬럼을 가리킨다. 소문자로 `"unknown"` 이라고 기재했다면 다른 이름이 되어 Oracle 에서 컬럼을 찾지 못한다.

외래 키와 관계

차원 테이블의 외래 키 컬럼은 [oracle-foreign-keys] 에서 본 규칙대로 두 번 매핑한다. `CUSTOMERS.COUNTRY_ID` 라면 `MappedCustomer` 의 읽기 전용 속성 `countryId` 가 컬럼을 읽고, 엔티티 `Customer` 의 다대일 관계 `country` 가 컬럼에 쓴다. `countryId` 의 getter 는 `public` 이고 setter 는 `protected` 다. `Customer` 의 `setCountry(Country)` 는 관계를 바꾸면서 `countryId` 도 그 나라의 식별자로 바꾸므로, 영속화하기 전에도 두 속성의 값이 어긋나지 않는다. 외래 키 컬럼이 식별자에 들어 있는 두 사실 테이블은 다르게 매핑한다(기본 키가 없는 사실 테이블).

관계는 차원으로 가는 쪽에만 둔다. `CUSTOMERS` 에서 `SALES` 로 가는 관계는 매핑하지 않았다. 고객 한 명의 판매를 모두 컬렉션으로 읽는 일은 드물고, 필요하면 쿼리로 읽는다. Exercises 절의 연습문제가 그렇게 한다.

`SUPPLEMENTARY_DEMOGRAPHICS` 의 기본 키 `CUST_ID` 는 `CUSTOMERS` 의 `CUST_ID` 와 같은 값이다. 행 4,500 개의 `CUST_ID` 가 모두 `CUSTOMERS` 에 있지만, 그것을 지키는 외래 키는 없다. 데이터 사전에는 기본 키와 NOT NULL 제약이 있고, 외래 키는 없다. 엔티티 `SupplementaryDemographics` 에도 `CUSTOMERS` 로 가는 관계는 두지 않았다.

판촉이 없는 판매

스키마를 만드는 스크립트의 주석은 PROMOTIONS 와 사실 테이블 사이에 외래 키가 없다고 한다. 외부 조인을 보이려고 일부러 두지 않았다는 것이다. 그러나 같은 스크립트가 SALES 와 COSTS 에 PROMOTIONS 를 참조하는 외래 키를 만들고, 데이터 사전에도 그 외래 키가 있다. 주석이 스크립트의 변경을 반영하지 못한 것이다.

판촉은 503 개나 있지만, 실제 판매에 쓰인 판촉은 넷뿐이다.

판매에 쓰인 판촉

```
  PROMO_ID PROMO_NAME                 PROMO_SUBCATEGORY      PROMO_CATEGORY
---------- -------------------------- ---------------------- --------------
        33 post promotion #20-33      downtown billboard     post
       350 internet promotion #29-350 online discount        internet
       351 TV promotion #13-351       TV program sponsorship TV
       999 NO PROMOTION #             NO PROMOTION           NO PROMOTION
```

999 는 판촉이 없었다는 뜻의 행이다. 판촉 없이 판 것도 PROMO_ID 가 NULL 이 아니라 999 이므로, SALES 의 PROMO_ID 는 NOT NULL 이다. 판매 918,843 건 가운데 887,837 건이 999 다. 판촉으로 판 것을 찾으려면 PROMO_CATEGORY 가 NO PROMOTION 이 아닌 판매를 찾는다. Exercises 절의 연습문제가 이 조건을 쓴다.

기본 키가 없는 사실 테이블

두 사실 테이블 SALES 와 COSTS 에는 기본 키가 없다. 외래 키 컬럼을 묶어 식별자로 정한 방법과, 같은 조합의 행이 둘이 되면 둘째 행의 값을 잃는다는 한계는 식별자를 정하는 방법 에서 보았다. 이 프로젝트의 문서는 Hibernate ORM 과 EclipseLink 가 모두 그렇게 동작한다고 기재한다.

복합 기본 키는 임베더블 클래스로 매핑하거나, 식별자 클래스를 두고 엔티티의 여러 속성으로 매핑한다.[11] Sales 는 앞의 방법을, Cost 는 뒤의 방법을 쓴다. 식별자에 든 외래 키 컬럼을 누가 쓰는지도 둘이 반대다.

SALES 와 @EmbeddedId

SALES 의 식별자는 다섯 외래 키 컬럼이다. 다섯 컬럼으로 묶어 두 번 이상 나오는 조합을 세는 쿼리와 그 결과는 아래와 같다.

다섯 컬럼으로 묶어 두 번 이상 나오는 조합

```
SELECT PROD_ID, CUST_ID, TIME_ID, CHANNEL_ID, PROMO_ID, COUNT(1) c
FROM SALES
GROUP BY PROD_ID, CUST_ID, TIME_ID, CHANNEL_ID, PROMO_ID
HAVING COUNT(1) > 1
;
```

결과

```
no rows selected
```

Sales 는 다섯 컬럼을 식별자 클래스 SalesId 로 묶어 @EmbeddedId 로 매핑한다. 이 식별자는 매핑된 슈퍼클래스 MappedSales 가 아니라 엔티티 Sales 가 선언하고, MappedSales 는 추상 메서드 getId_() 로 그 값을 읽는다.

Sales 와 id_

```
@Entity
@Table(name = MappedSales.TABLE_NAME,
       comment = "facts table, without a primary key; all rows are uniquely identified by the " +
               "combination of all foreign keys")
public class Sales
        extends MappedSales {

    // @NotNull 이 없다: 구현체는 flush 할 때 @MapsId 관계에서 이 값을 채우는데, 그것은 EclipseLink 가
    // 검증하는 prePersist 보다 늦다. 이 값의 속성은 저마다 여전히 @NotNull 이고, 그 값을 쓰는
    // 관계도 그렇다.
    @Valid
    @EmbeddedId
    private SalesId id_;
}
```

`@EmbeddedId` 로 매핑한 식별자는 엔티티의 속성 하나이고, 그 형이 식별자 클래스다.[12]

SalesId

```
@Embeddable
public class SalesId {

    @NotNull
    @Max(MappedSales.ATTRIBUTE_MAX_PROD_ID)
    @Min(MappedSales.ATTRIBUTE_MIN_PROD_ID)
    @Basic(optional = false)
    @Column(name = MappedSales.COLUMN_NAME_PROD_ID, nullable = false, insertable = true,
            updatable = false,
            comment = "FK to the products dimension table")
    private Integer prodId;

    @NotNull
    @Basic(optional = false)
    @Column(name = MappedSales.COLUMN_NAME_CUST_ID, nullable = false, insertable = true,
            updatable = false,
            comment = "FK to the customers dimension table")
    private Long custId;

    @NotNull
    @Basic(optional = false)
    @Column(name = MappedSales.COLUMN_NAME_TIME_ID, nullable = false, insertable = true,
            updatable = false,
            comment = "FK to the times dimension table")
    private LocalDateTime timeId;

    @NotNull
    @Max(MappedSales.ATTRIBUTE_MAX_CHANNEL_ID)
    @Min(MappedSales.ATTRIBUTE_MIN_CHANNEL_ID)
    @Basic(optional = false)
    @Column(name = MappedSales.COLUMN_NAME_CHANNEL_ID, nullable = false, insertable = true,
            updatable = false,
            comment = "FK to the channels dimension table")
    private Long channelId;

    @NotNull
    @Max(MappedSales.ATTRIBUTE_MAX_PROMO_ID)
    @Min(MappedSales.ATTRIBUTE_MIN_PROMO_ID)
    @Basic(optional = false)
    @Column(name = MappedSales.COLUMN_NAME_PROMO_ID, nullable = false, insertable = true,
            updatable = false,
            comment = "promotion identifier, without FK constraint (intentionally) to show outer " +
                    "join optimization")
    private Integer promoId;
}
```

다섯 속성이 다섯 컬럼을 매핑한다. `updatable` 은 모두 `false` 다. 식별자의 값은 영속화한 뒤에 바꾸면 안 되기 때문이다.[13]

`channelId` 가 `NUMBER(1)` 인데도 `Long` 이고 `prodId` 와 `promoId` 가 `Integer` 인 이유는 수 에서 보았다. `@Min` 과 `@Max` 는 컬럼의 정밀도를 따른다.

SalesId 의 equals 와 hashCode 는 다섯 값을 모두 쓴다.[14] 그런데 MappedSales 의 hashCode 는 모든 인스턴스에서 같은 값을 돌려준다. 식별자는 영속화할 때에야 채워지는데, 해시를 쓰는 컬렉션에 들어 있는 동안 해시 값이 바뀌면 안 되기 때문이다.

관계에서 가져오는 식별자

다섯 컬럼은 외래 키이기도 하다. Sales 는 다섯 컬럼마다 차원의 엔티티로 가는 다대일 관계를 하나씩 두고, 관계마다 @MapsId 로 식별자의 속성 하나를 기재한다.

Sales 의 product

```
@Valid
@NotNull
@MapsId(SalesId.ATTRIBUTE_NAME_PROD_ID) ❶
@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = COLUMN_NAME_PROD_ID,
            referencedColumnName = Product.COLUMN_NAME_PROD_ID,
            nullable = false,
            insertable = true,          ❷
            updatable = false)
private Product product;
```

❶ 식별자 id_ 의 prodId 는 이 관계가 참조하는 Product 의 식별자에서 가져온다.

❷ 컬럼은 이 관계가 쓴다. updatable 은 false 이므로, 영속화한 뒤에는 바꾸지 않는다.

Sales.of(product, customer, time, channel, promotion) 는 다섯 관계만 받아 판매를 생성한다. 식별자는 그때 null 이고, 영속화할 때 구현체가 다섯 관계에서 채운다. 그래서 id_ 에는 @NotNull 을 붙이지 않고, 값을 쓰는 다섯 관계에 @NotNull 을 붙인다. 영속화하기 전에 검증하는 시점에는 식별자가 아직 null 이기 때문이다.

관계가 있으므로 Query Language 에서는 s.product 처럼 관계를 따라가 차원으로 조인할 수 있다. 식별자의 값은 s.id_.prodId 처럼 식별자를 거쳐 가리킨다.

SALES 는 TIME_ID 의 범위로 나눈 파티션 테이블이다(파티션). 2020 년 1 분기의 파티션 SALES_Q1_2020 에는 64,186 건이 들어 있다. s.id_.timeId 나 s.time.timeId 에 거는 조건은 SQL 에서 같은 TIME_ID 의 조건이 된다.

SalesUtils

SalesUtils 는 Sales 를 다루는 정적 메서드를 모은다. 식별자의 다섯 값을 하나씩 읽는 메서드와 함께, 판매 금액의 통계를 쿼리 한 번으로 구하는 메서드가 있다. 식별자는 구현체가 관계에서 채우므로, 식별자의 값을 넣는 메서드는 없다.

SalesUtils 의 getAmountSoldStatistics

```
public static AmountSoldStatistics getAmountSoldStatistics(
        final EntityManager entityManager,
        final BiFunction<? super CriteriaBuilder, ? super Root<Sales>, ? extends Predicate> where) {
    Objects.requireNonNull(entityManager, "entityManager is null");
    final var builder = entityManager.getCriteriaBuilder();
    final var query = builder.createTupleQuery();
    final var root = query.from(Sales.class);
    final var path = root.get(Sales_.amountSold);
    query.select(builder.tuple(builder.count(root), builder.sum(path), builder.min(path), ❶
                               builder.max(path)));
    if (where != null) {
        query.where(where.apply(builder, root));                                    ❷
    }
    final var tuple = entityManager.createQuery(query).getSingleResult();
    final var count = ((Number) tuple.get(0)).longValue();
    if (count == 0L) {
        return new AmountSoldStatistics(0L, BigDecimal.ZERO, null, null);
    }
    return new AmountSoldStatistics(count, decimal(tuple.get(1)), decimal(tuple.get(2)),
                                    decimal(tuple.get(3)));
}
```

❶ 건수, 합계, 최소, 최대를 한 행으로 읽는다.

❷ 어느 판매를 셀지는 호출하는 쪽이 조건으로 넘긴다. null 이면 모든 판매를 센다.

AmountSoldStatistics 는 건수와 합계, 최소, 최대를 담는 레코드다. JDK 의 LongSummaryStatistics 에 해당하는 것이 BigDecimal 에는 없으므로 따로 만들었다. 918,843 행을 엔티티로 읽어 자바에서 더하지 않고, 데이터베이스가 집계한 한 행만 읽는다. decimal(…) 은 구현체가 집계의 결과를 어떤 수의 형으로 주든 BigDecimal 로 바꾼다.

COSTS 와 @IdClass

COSTS 의 식별자는 고객을 뺀 네 외래 키 컬럼이다. 뷰 PROFITS 절의 뷰 PROFITS 가 SALES 와 COSTS 를 조인하는 것도 이 네 컬럼이다. Cost 는 @Id 속성을 넷 두고, 그 네 값을 담는 식별자 클래스 CostId 를 @IdClass 로 기재한다.[15] @IdClass 는 엔티티 Cost 에 있고, 네 @Id 속성은 매핑된 슈퍼클래스 MappedCost 에 있다.

Cost

```
@Entity
@IdClass(CostId.class) ❶
@Table(name = MappedCost.TABLE_NAME)
public class Cost
        extends MappedCost { }
```

❶ 식별자 클래스는 CostId 다.

MappedCost 의 네 @Id 속성

```
@Id
@NotNull
@Basic(optional = false)
@Column(name = COLUMN_NAME_PROD_ID, nullable = false, insertable = true, updatable = false)
private Integer prodId;

@Id
@NotNull
@Basic(optional = false)
@Column(name = COLUMN_NAME_TIME_ID, nullable = false, insertable = true, updatable = false)
private LocalDateTime timeId;

@Id
@NotNull
@Basic(optional = false)
@Column(name = COLUMN_NAME_PROMO_ID, nullable = false, insertable = true, updatable = false)
private Integer promoId;

@Id
@NotNull
@Basic(optional = false)
@Column(name = COLUMN_NAME_CHANNEL_ID, nullable = false, insertable = true, updatable = false)
private Long channelId;
```

네 속성은 외래 키이지만, 다른 테이블의 외래 키 속성과 달리 컬럼을 쓴다(`insertable = true`). getter 와 setter 는 모두 `protected` 다.

`Sales` 에서는 식별자의 값을 `s.id_.timeId` 로 가리켰지만, `Cost` 에서는 `c.timeId` 로 가리킨다. 식별자의 값이 엔티티의 속성이기 때문이다.

식별자 클래스의 속성은 엔티티의 `@Id` 속성과 이름과 형이 같아야 한다.[16]

CostId

```
public class CostId {

    @NotNull
    private Integer prodId;

    @NotNull
    private LocalDateTime timeId;

    @NotNull
    private Integer promoId;

    @NotNull
    private Long channelId;
}
```

`CostId` 는 컬럼을 매핑하지 않는 평범한 클래스다. 속성의 이름과 형이 `MappedCost` 의 `@Id` 속성과 같고, 검증 제약 `@NotNull` 만 붙어 있다. 식별자 클래스에 매핑이 꼭 있어야 하는 것은 아니다. [film_category-filmcategory] 절의 `FilmCategoryId` 에도 매핑 애너테이션이 하나도 없었다. 컬럼을 `@Column` 으로 매핑하는 `SalesId` 와 비교해 보면 두 방법의 차이가 드러난다.

식별자로 찾을 때에는 네 값을 넣은 CostId 를 EntityManager.find 에 넘긴다.

관계는 읽기만 한다

네 컬럼 위의 다대일 관계는 Sales 와 반대다. 컬럼은 @Id 속성이 쓰고, 관계는 insertable = false, updatable = false 로 같은 컬럼을 읽기만 한다.

Cost 의 product 와 setProduct

```
private void setProduct(final Product product) {
    this.product = product;
    setProdId(Optional.ofNullable(product).map(Product::getProdId).orElse(null)); ❶
}

@Valid
@NotNull
@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = COLUMN_NAME_PROD_ID,
            referencedColumnName = Product.COLUMN_NAME_PROD_ID,
            nullable = false,
            insertable = false,                                    ❷
            updatable = false)
private Product product;
```

❶ 관계를 정하면서 관계의 식별자를 @Id 속성에 복사한다.

❷ 컬럼은 prodId 가 쓰고, 관계는 읽기만 한다.

Cost.of(product, time, promotion, channel) 는 네 관계를 받아 원가를 생성하고, 관계마다 private setter 가 식별자의 값을 함께 채운다. 그래서 영속화하기 전에 검증하는 시점에도 관계와 @Id 속성이 모두 채워져 있고, 관계에도 @NotNull 을 붙일 수 있다.

@Id 를 관계에 붙여 식별자를 관계로 매핑할 수도 있다. FilmCategory 가 category 에 그렇게 했다. Cost 는 네 컬럼을 모두 기본 속성으로 쓰고 관계는 읽기만 하므로, CostId 의 속성도 모두 기본 형이다.

뷰 PROFITS

PROFITS 는 판매마다 원가(原價, cost)를 붙여 보여 주는 뷰다. SALES 와 COSTS 를 네 컬럼으로 조인하고, 원가에 수량을 곱한 총원가 TOTAL_COST 를 더한다.

PROFITS 뷰의 정의

```
SELECT
  s.channel_id,
  s.cust_id,
  s.prod_id,
  s.promo_id,
  s.time_id,
  c.unit_cost,
  c.unit_price,
  s.amount_sold,
  s.quantity_sold,
  c.unit_cost * s.quantity_sold TOTAL_COST
 FROM
  costs c, sales s
 WHERE c.prod_id = s.prod_id
   AND c.time_id = s.time_id
   AND c.channel_id = s.channel_id
   AND c.promo_id = s.promo_id
```

뷰의 행은 916,039 개다. 판매 918,843 건 가운데 같은 상품, 날, 경로, 판촉의 원가가 없는 2,804 건은 내부 조인에서 빠진다. 뷰의 컬럼은 원래 테이블의 NOT NULL 을 물려받았고, 계산한 TOTAL_COST 만 NULL 을 허용한다.

식별자

뷰에도 키가 없다. SALES 의 한 행은 COSTS 의 행 하나와만 조인되므로, 뷰의 행은 SALES 의 행마다 많아야 하나다. 그래서 SALES 의 식별자였던 다섯 컬럼이 뷰에서도 행을 식별한다(식별자를 정하는 방법). 다만 이것은 SALES 와 COSTS 의 식별자가 유일할 때만 성립하고, 그 유일성은 데이터베이스가 보장하지 않는다.

Profit 은 Cost 처럼 @IdClass 로 다섯 컬럼의 식별자를 매핑한다. 식별자 클래스는 ProfitId 이고, 다섯 @Id 속성은 매핑된 슈퍼클래스 MappedProfit 에 있다.

Profit

```
@Entity
@IdClass(ProfitId.class)
@Table(name = MappedProfit.TABLE_NAME)
public class Profit
        extends MappedProfit { }
```

뷰의 식별자도 insertable 이다

MappedProfit 의 channelId

```
@Id
@NotNull
@Max(ATTRIBUTE_MAX_CHANNEL_ID)
@Min(ATTRIBUTE_MIN_CHANNEL_ID)
@Basic(optional = false)
// insertable=true 는 JPA 의 기본값이다; 굳이 기재하는 것은 EclipseLink 가 @Id 의
// insertable=false 를 거부하기 때문이다 -- "There should be one non-read-only mapping defined for the
// primary key field", EclipseLink-46. 어차피 뷰에는 아무것도 쓰지 않는다.
@Column(name = COLUMN_NAME_CHANNEL_ID, nullable = false, insertable = true, updatable = false) ❶
private Long channelId;
```

❶ 뷰의 컬럼이지만 `insertable` 은 `true` 다.

뷰에는 아무것도 쓰지 않으므로, 모든 컬럼을 `insertable = false`, `updatable = false` 로 매핑하는 것이 자연스럽다. **그런데 EclipseLink 는 식별자의 컬럼을 쓰는 매핑이 하나도 없으면 매핑을 거부한다.** 그래서 `@Id` 속성만은 `insertable = true` 로 둔다. `updatable` 은 `false` 이므로, 식별자는 `INSERT` 에만 들어갈 수 있다. `Profit` 을 영속화하는 코드가 없는 한 `INSERT` 는 일어나지 않는다.

식별자가 아닌 속성은 모두 `insertable = false`, `updatable = false` 로 읽기만 하고, `MappedProfit` 에는 getter 만 있고 setter 가 없다.

`Sales` 와 달리 `Profit` 에는 차원으로 가는 관계가 없다. 차원의 값이 필요하면 Query Language 에서 `p.prodId = pr.prodId` 처럼 식별자의 속성으로 조인한다.

구체화된 뷰

구체화된 뷰를 엔티티로 매핑하는 방법과 설치한 SH 의 두 구체화된 뷰가 비어 있는 이유는 [oracle-materialized-views] 에서 보았다. 식별자는 정의한 쿼리가 묶는 컬럼이다(식별자를 정하는 방법). 여기서는 두 구체화된 뷰의 정의와, 식별자를 매핑하며 생기는 일을 본다.

CAL_MONTH_SALES_MV

`CAL_MONTH_SALES_MV` 는 달마다 판매 금액을 합해 두는 구체화된 뷰다. 918,843 건의 판매를 달마다 다시 더하지 않고, 더해 둔 결과를 읽는다.

CAL_MONTH_SALES_MV 의 정의

```
CREATE MATERIALIZED VIEW cal_month_sales_mv
--  ON PREBUILT TABLE
   ENABLE QUERY REWRITE
   AS
   SELECT   t.calendar_month_desc,
            SUM(s.amount_sold) AS dollars
   FROM     sh.sales s,
            sh.times t
   WHERE    s.time_id = t.time_id
   GROUP BY t.calendar_month_desc;
```

`SALES` 와 `TIMES` 를 조인해 `TIMES` 의 `CALENDAR_MONTH_DESC` 로 묶고, 묶음마다 `AMOUNT_SOLD` 를 더한다.

Attribute Mappings

구체화된 뷰가 비어 있으므로, 갱신하면 무엇이 들어갈지는 정의한 쿼리를 그대로 실행해 보면 안다. 처음 세 달은 아래와 같고, 모두 48 행이다.

정의한 쿼리의 처음 세 행

```
CALENDAR    DOLLARS
-------- ----------
2019-01  2277420.49
2019-02  2372690.87
2019-03  1830572.64
```

정의한 쿼리가 `CALENDAR_MONTH_DESC` 로 묶으므로 한 달에 한 행만 있고, 이 컬럼이 식별자다.

MappedCalMonthSalesMv 의 calendarMonthDesc 와 dollars

```
@Size(max = SIZE_MAX_CALENDAR_MONTH_DESC)
@NotNull
@Id
@Basic(optional = false)
@Column(name = COLUMN_NAME_CALENDAR_MONTH_DESC,
        nullable = false,
        insertable = true, // eclipselink
        updatable = false,
        length = COLUMN_LENGTH_CALENDAR_MONTH_DESC)
private String calendarMonthDesc;

@Basic(optional = true)
@Column(name = COLUMN_NAME_DOLLARS, nullable = true, insertable = false, updatable = false) ❶
private BigDecimal dollars;
```

❶ 식별자가 아닌 속성은 읽기 전용이다.

식별자의 `insertable = true` 에 붙은 주석 `eclipselink` 는 뷰 PROFITS 절의 `Profit` 과 같은 이유다.

식별자에는 변환기를 쓸 수 없다

`CALENDAR_MONTH_DESC` 의 값은 `2019-01` 같은 문자열이다. 문자열과 수로 기재한 날짜 절에서 본 `OfYearMonth` 로 `YearMonth` 로 바꾸어 읽으면 좋겠지만, 이 속성은 식별자다. **명세는 식별자의 속성에 변환기를 적용하는 것을 지원하지 않는다.**[17] 자동으로 적용하는 변환기는 적용되지 않고, `@Convert` 로 적용하면 이식성이 없다.

그래서 속성은 `String` 으로 두고, `YearMonth` 가 필요하면 `CalMonthSalesMvUtils` 로 바꾼다.

CalMonthSalesMvUtils 의 getCalendarMonthDescAsYearMonth

```
public static YearMonth getCalendarMonthDescAsYearMonth(final CalMonthSalesMv calMonthSalesMv) {
    return getCalendarMonthDescAsMapped(calMonthSalesMv, YearMonth::parse);
}
```

`TimeUtils` 와 달리 값을 넣는 메서드는 없다. 식별자이고, 쓸 일도 없기 때문이다.

`yyyy-MM` 꼴의 문자열은 사전 순서와 시간 순서가 같다. 그래서 Query Language 에서도 `m.calendarMonthDesc BETWEEN '2021-01' AND '2021-12'` 처럼 문자열로 기간을 걸 수 있다.

FWEEK_PSCAT_SALES_MV

FWEEK_PSCAT_SALES_MV 는 주와 상품의 하위 분류, 판매 경로, 판촉마다 판매 금액을 합해 두는 구체화된 뷰다.

FWEEK_PSCAT_SALES_MV 의 정의

```
CREATE MATERIALIZED VIEW fweek_pscat_sales_mv
--  ON PREBUILT TABLE
   ENABLE QUERY REWRITE
   AS
   SELECT   t.week_ending_day,
            p.prod_subcategory,
            SUM(s.amount_sold) AS dollars,
            s.channel_id,
            s.promo_id
   FROM     sh.sales s,
            sh.times t,
            sh.products p
   WHERE    s.time_id = t.time_id
      AND   s.prod_id = p.prod_id
   GROUP BY t.week_ending_day,
            p.prod_subcategory,
            s.channel_id,
            s.promo_id;
```

SALES 에 TIMES 와 PRODUCTS 를 조인해, 주의 마지막 날 WEEK_ENDING_DAY, 하위 분류의 이름 PROD_SUBCATEGORY, CHANNEL_ID, PROMO_ID 의 네 컬럼으로 묶는다. 쌍이 어긋난 행 에서 본 대로 하위 분류의 이름과 번호는 어긋나 있는데, 이 구체화된 뷰는 이름으로 묶는다.

이 구체화된 뷰도 비어 있으므로, 정의한 쿼리를 그대로 실행했다. 10,512 행이 나오고, 처음 세 행은 아래와 같다.

정의한 쿼리의 처음 세 행

```
WEEK_ENDING_DAY PROD_SUBCATEGORY        DOLLARS CHANNEL_ID   PROMO_ID
--------------- -------------------- ---------- ---------- ----------
2019-01-06      Baseball Clothing        341.66          2        999
2019-01-06      Baseball Clothing        385.47          3        999
2019-01-06      Baseball Clothing        246.88          4        999
```

GROUP BY 의 네 컬럼은 모두 NOT NULL 이므로 식별자가 된다. FweekPscatSalesMv 는 네 컬럼을 Sales 처럼 @EmbeddedId 로 매핑하고, 이 식별자도 매핑된 슈퍼클래스가 아니라 엔티티가 선언한다.

FweekPscatSalesMv

```
@Entity
@Table(name = MappedFweekPscatSalesMv.TABLE_NAME)
public class FweekPscatSalesMv
        extends MappedFweekPscatSalesMv {

    @Valid
    @NotNull
    @EmbeddedId
    private FweekPscatSalesMvId id_;
}
```

Sales 와 달리 식별자를 관계에서 가져오지 않으므로, id_ 에 @NotNull 을 붙인다.

FweekPscatSalesMvId 의 weekEndingDay

```
@Embeddable
public class FweekPscatSalesMvId {

    // insertable=true 는 JPA 의 기본값이다; 굳이 기재하는 것은 EclipseLink 가 읽기 전용이 아닌 매핑이
    // 하나도 없는 식별자를 거부하기 때문이다 -- "There should be one non-read-only mapping defined for the
    // primary key field", EclipseLink-46. 어차피 뷰에는 아무것도 쓰지 않는다.

    @NotNull
    @Basic(optional = false)
    @Column(name = MappedFweekPscatSalesMv.COLUMN_NAME_WEEK_ENDING_DAY,
            nullable = false,
            insertable = true,
            updatable = false)
    private LocalDateTime weekEndingDay;
}
```

MappedProfit 과 MappedCalMonthSalesMv 처럼, 식별자의 네 속성은 insertable = true, updatable = false 다. channelId 와 promoId 의 형은 SalesId 와 같은 Long 과 Integer 다. 식별자가 아닌 속성은 dollars 하나이고, 읽기 전용이다.

@EmbeddedId 로 매핑하면 식별자의 값을 읽을 때 getId_() 를 거쳐야 하고, 식별자가 null 이면 그것도 확인해야 한다. FweekPscatSalesMvUtils 는 식별자의 네 값을 하나씩 읽는 정적 메서드를 모은다. SALES 와 @EmbeddedId 절의 SalesUtils 에도 같은 메서드가 있다.

Profit 처럼 이 엔티티에도 차원으로 가는 관계가 없다. 하위 분류의 이름으로 묶었으므로 PRODUCTS 의 한 행을 가리키지도 않는다. 판매 경로와 판촉이 필요하면 f.id.channelId = c.channelId 처럼 식별자의 속성으로 조인한다.

Exercises

이 장의 데이터로 풀 수 있는 연습문제 가운데 이 장의 매핑을 연습하기 좋은 문제를 선택해 모았다. 문제마다 Query Language 와 Criteria API 로 푼 답을 함께 실었다. 풀이는 프로젝트의 `src/test/sql-exercises` 에 있는 SQL 파일과, 같은 문제를 Query Language 와 Criteria API 로 푼 통합 테스트에 있다.

SH_SCHEMA_A_02

두 연도를 받아, 앞 연도에는 샀지만 뒤 연도에는 아무것도 사지 않은 고객을 구한다.

`EXCEPT` 는 앞 쿼리의 결과에서 뒤 쿼리의 결과에 있는 것을 뺀다. Jakarta Persistence 3.2 부터 Query Language 에서 `EXCEPT` 를, `CriteriaBuilder` 에서 `except()` 를 쓸 수 있다. 두 쿼리는 연도만 다르므로, Criteria API 의 답은 한 해에 산 고객을 구하는 쿼리를 메서드 `customersOf()` 로 생성하고 그 결과 두 개를 `except()` 에 넘긴다.

Query Language

```
SELECT cu
FROM Sales s
         JOIN s.customer cu
         JOIN s.time t
WHERE t.calendarYear = :previousYear
EXCEPT
SELECT cu2
FROM Sales s2
         JOIN s2.customer cu2
         JOIN s2.time t2
WHERE t2.calendarYear = :currentYear
```

Criteria API, 한 해에 산 고객을 구하는 쿼리

```
private static CriteriaQuery<Customer> customersOf(final CriteriaBuilder cb,
                                                   final String yearParameterName) {
    final var cq = cb.createQuery(Customer.class);
    final var s = cq.from(Sales.class);
    cq.select(s.join(Sales_.customer))
            .where(cb.equal(s.join(Sales_.time).get(Time_.calendarYear),
                            cb.parameter(Integer.class, yearParameterName)));
    return cq;
}
```

Criteria API

```
final var cb = em.getCriteriaBuilder();
final var cs = cb.except(customersOf(cb, "previousYear"),
                         customersOf(cb, "currentYear"));
return em.createQuery(cs)
        .setParameter("previousYear", PREVIOUS_YEAR)
        .setParameter("currentYear", CURRENT_YEAR)
        .getResultList();
```

두 제공자 모두 두 API 로 같은 SQL 을 보낸다.

Hibernate ORM 이 보내는 SQL

```
select c1_0.CUST_ID, c1_0.COUNTRY_ID, c1_0.CUST_CITY, c1_0.CUST_CITY_ID, c1_0.CUST_CREDIT_LIMIT,
c1_0.CUST_EFF_FROM, c1_0.CUST_EFF_TO, c1_0.CUST_EMAIL, c1_0.CUST_FIRST_NAME, c1_0.CUST_GENDER,
c1_0.CUST_INCOME_LEVEL, c1_0.CUST_LAST_NAME, c1_0.CUST_MAIN_PHONE_NUMBER, c1_0.CUST_MARITAL_STATUS,
c1_0.CUST_POSTAL_CODE, c1_0.CUST_SRC_ID, c1_0.CUST_STATE_PROVINCE, c1_0.CUST_STATE_PROVINCE_ID,
c1_0.CUST_STREET_ADDRESS, c1_0.CUST_TOTAL, c1_0.CUST_TOTAL_ID, c1_0.CUST_VALID, c1_0.CUST_YEAR_OF_BIRTH
from SH.SALES s1_0
join SH.CUSTOMERS c1_0 on c1_0.CUST_ID=s1_0.CUST_ID
join SH.TIMES t1_0 on t1_0.TIME_ID=s1_0.TIME_ID
where t1_0.CALENDAR_YEAR=?
except
select c3_0.CUST_ID, c3_0.COUNTRY_ID, c3_0.CUST_CITY, c3_0.CUST_CITY_ID, c3_0.CUST_CREDIT_LIMIT,
c3_0.CUST_EFF_FROM, c3_0.CUST_EFF_TO, c3_0.CUST_EMAIL, c3_0.CUST_FIRST_NAME, c3_0.CUST_GENDER,
c3_0.CUST_INCOME_LEVEL, c3_0.CUST_LAST_NAME, c3_0.CUST_MAIN_PHONE_NUMBER, c3_0.CUST_MARITAL_STATUS,
c3_0.CUST_POSTAL_CODE, c3_0.CUST_SRC_ID, c3_0.CUST_STATE_PROVINCE, c3_0.CUST_STATE_PROVINCE_ID,
c3_0.CUST_STREET_ADDRESS, c3_0.CUST_TOTAL, c3_0.CUST_TOTAL_ID, c3_0.CUST_VALID, c3_0.CUST_YEAR_OF_BIRTH
from SH.SALES s2_0
join SH.CUSTOMERS c3_0 on c3_0.CUST_ID=s2_0.CUST_ID
join SH.TIMES t2_0 on t2_0.TIME_ID=s2_0.TIME_ID
where t2_0.CALENDAR_YEAR=?
```

Hibernate ORM 은 `except` 를 그대로 보내고, EclipseLink 는 Oracle 의 `MINUS` 로 보낸다. EclipseLink 는 이 쿼리 뒤에 고객의 나라를 기본 키로 하나씩 읽는 SELECT 를 16개 더 보낸다. 위빙 없이 실행해서 `LAZY` 로 매핑한 다대일 관계도 즉시 읽기 때문이다.

EclipseLink 가 보내는 SQL

```
(SELECT t0.CUST_ID, t0.COUNTRY_ID, t0.CUST_CITY, t0.CUST_CITY_ID, t0.CUST_CREDIT_LIMIT, t0.CUST_EFF_FROM,
t0.CUST_EFF_TO, t0.CUST_EMAIL, t0.CUST_FIRST_NAME, t0.CUST_GENDER, t0.CUST_INCOME_LEVEL, t0.CUST_LAST_NAME,
t0.CUST_MAIN_PHONE_NUMBER, t0.CUST_MARITAL_STATUS, t0.CUST_POSTAL_CODE, t0.CUST_SRC_ID,
t0.CUST_STATE_PROVINCE, t0.CUST_STATE_PROVINCE_ID, t0.CUST_STREET_ADDRESS, t0.CUST_TOTAL, t0.CUST_TOTAL_ID,
t0.CUST_VALID, t0.CUST_YEAR_OF_BIRTH
FROM SH.CUSTOMERS t0, SH.TIMES t2, SH.SALES t1
WHERE ((t2.CALENDAR_YEAR = ?) AND ((t2.TIME_ID = t1.TIME_ID) AND (t0.CUST_ID = t1.CUST_ID)))
MINUS (SELECT t3.CUST_ID, t3.COUNTRY_ID, t3.CUST_CITY, t3.CUST_CITY_ID, t3.CUST_CREDIT_LIMIT,
t3.CUST_EFF_FROM, t3.CUST_EFF_TO, t3.CUST_EMAIL, t3.CUST_FIRST_NAME, t3.CUST_GENDER, t3.CUST_INCOME_LEVEL,
t3.CUST_LAST_NAME, t3.CUST_MAIN_PHONE_NUMBER, t3.CUST_MARITAL_STATUS, t3.CUST_POSTAL_CODE, t3.CUST_SRC_ID,
t3.CUST_STATE_PROVINCE, t3.CUST_STATE_PROVINCE_ID, t3.CUST_STREET_ADDRESS, t3.CUST_TOTAL, t3.CUST_TOTAL_ID,
t3.CUST_VALID, t3.CUST_YEAR_OF_BIRTH
FROM SH.TIMES t5, SH.SALES t4, SH.CUSTOMERS t3
WHERE ((t5.CALENDAR_YEAR = ?) AND ((t5.TIME_ID = t4.TIME_ID) AND (t3.CUST_ID = t4.CUST_ID))))
bind => [2020, 2021]
```

쿼리는 1,269행을 돌려주고, 테스트는 앞의 10행만 로그에 남긴다. 집합 연산의 결과에는 순서가 없어서, Criteria API 로 실행했을 때는 다른 고객이 먼저 나왔다.

로그에 남긴 결과

```
com.github.jinahya.oracle.sample.persistence.sh.Customer@c91{custId=3217,custFirstName=Zenda,custLastName=Ma
rkerman,...
com.github.jinahya.oracle.sample.persistence.sh.Customer@428{custId=1064,custFirstName=Sanders,custLastName=
Greenley,...
com.github.jinahya.oracle.sample.persistence.sh.Customer@2906{custId=10502,custFirstName=Gwendolyn,custLastN
ame=Lawson,...
...
com.github.jinahya.oracle.sample.persistence.sh.Customer@1595{custId=5525,custFirstName=Roy,custLastName=Sto
ckman,...
```

SH_SCHEMA_A_03

연도를 받아, 그해 `Internet` 과 `Direct Sales` 두 판매 경로로 모두 산 고객을 구한다.

`INTERSECT` 는 두 쿼리의 결과에 모두 있는 것만 남긴다. `EXCEPT` 와 마찬가지로 Jakarta Persistence 3.2 부터 Query Language 와 `CriteriaBuilder` 에서 쓸 수 있다. Criteria API 의 답은 한 판매 경로로 산 고객을 구하는 쿼리를 메서드 `customersThrough()` 로 생성하고, 판매 경로의 매개변수 이름만 달리한 두 쿼리를 `intersect()` 에 넘긴다.

Query Language

```
SELECT cu
FROM Sales s
         JOIN s.customer cu
         JOIN s.time t
         JOIN s.channel ch
WHERE t.calendarYear = :year
  AND ch.channelDesc = :channelA
INTERSECT
SELECT cu2
FROM Sales s2
         JOIN s2.customer cu2
         JOIN s2.time t2
         JOIN s2.channel ch2
WHERE t2.calendarYear = :year
  AND ch2.channelDesc = :channelB
```

Criteria API, 한 판매 경로로 산 고객을 구하는 쿼리

```
private static CriteriaQuery<Customer> customersThrough(final CriteriaBuilder cb,
                                                         final String channelParameterName) {
    final var cq = cb.createQuery(Customer.class);
    final var s = cq.from(Sales.class);
    cq.select(s.join(Sales_.customer))
            .where(cb.equal(s.join(Sales_.time).get(Time_.calendarYear),
                            cb.parameter(Integer.class, "year")),
                   cb.equal(s.join(Sales_.channel).get(Channel_.channelDesc),
                            cb.parameter(String.class, channelParameterName)));
    return cq;
}
```

Criteria API

```
final var cb = em.getCriteriaBuilder();
final var cs = cb.intersect(customersThrough(cb, "channelA"),
                            customersThrough(cb, "channelB"));
return em.createQuery(cs)
        .setParameter("year", YEAR)
        .setParameter("channelA", CHANNEL_A)
        .setParameter("channelB", CHANNEL_B)
        .getResultList();
```

Hibernate ORM 은 두 API 로 같은 SQL 을 보낸다.

Hibernate ORM 이 보내는 SQL

```
select c1_0.CUST_ID, c1_0.COUNTRY_ID, c1_0.CUST_CITY, c1_0.CUST_CITY_ID, c1_0.CUST_CREDIT_LIMIT,
c1_0.CUST_EFF_FROM, c1_0.CUST_EFF_TO, c1_0.CUST_EMAIL, c1_0.CUST_FIRST_NAME, c1_0.CUST_GENDER,
c1_0.CUST_INCOME_LEVEL, c1_0.CUST_LAST_NAME, c1_0.CUST_MAIN_PHONE_NUMBER, c1_0.CUST_MARITAL_STATUS,
c1_0.CUST_POSTAL_CODE, c1_0.CUST_SRC_ID, c1_0.CUST_STATE_PROVINCE, c1_0.CUST_STATE_PROVINCE_ID,
c1_0.CUST_STREET_ADDRESS, c1_0.CUST_TOTAL, c1_0.CUST_TOTAL_ID, c1_0.CUST_VALID, c1_0.CUST_YEAR_OF_BIRTH
from SH.SALES s1_0
join SH.CUSTOMERS c1_0 on c1_0.CUST_ID=s1_0.CUST_ID
join SH.TIMES t1_0 on t1_0.TIME_ID=s1_0.TIME_ID
join SH.CHANNELS c2_0 on c2_0.CHANNEL_ID=s1_0.CHANNEL_ID
where t1_0.CALENDAR_YEAR=? and c2_0.CHANNEL_DESC=?
intersect
select c4_0.CUST_ID, c4_0.COUNTRY_ID, c4_0.CUST_CITY, c4_0.CUST_CITY_ID, c4_0.CUST_CREDIT_LIMIT,
c4_0.CUST_EFF_FROM, c4_0.CUST_EFF_TO, c4_0.CUST_EMAIL, c4_0.CUST_FIRST_NAME, c4_0.CUST_GENDER,
c4_0.CUST_INCOME_LEVEL, c4_0.CUST_LAST_NAME, c4_0.CUST_MAIN_PHONE_NUMBER, c4_0.CUST_MARITAL_STATUS,
c4_0.CUST_POSTAL_CODE, c4_0.CUST_SRC_ID, c4_0.CUST_STATE_PROVINCE, c4_0.CUST_STATE_PROVINCE_ID,
c4_0.CUST_STREET_ADDRESS, c4_0.CUST_TOTAL, c4_0.CUST_TOTAL_ID, c4_0.CUST_VALID, c4_0.CUST_YEAR_OF_BIRTH
from SH.SALES s2_0
join SH.CUSTOMERS c4_0 on c4_0.CUST_ID=s2_0.CUST_ID
join SH.TIMES t2_0 on t2_0.TIME_ID=s2_0.TIME_ID
join SH.CHANNELS c5_0 on c5_0.CHANNEL_ID=s2_0.CHANNEL_ID
where t2_0.CALENDAR_YEAR=? and c5_0.CHANNEL_DESC=?
```

EclipseLink 가 두 API 로 보내는 SQL 은 `TIMES` 와 `CHANNELS` 의 별칭 번호만 다르다. EclipseLink 는 이 쿼리 뒤에 고객의 나라를 기본 키로 하나씩 읽는 SELECT 를 13개 더 보낸다. 위빙 없이 실행해서 `LAZY` 로 매핑한 다대일 관계도 즉시 읽기 때문이다.

EclipseLink 가 보내는 SQL

```
(SELECT t0.CUST_ID, t0.COUNTRY_ID, t0.CUST_CITY, t0.CUST_CITY_ID, t0.CUST_CREDIT_LIMIT, t0.CUST_EFF_FROM,
t0.CUST_EFF_TO, t0.CUST_EMAIL, t0.CUST_FIRST_NAME, t0.CUST_GENDER, t0.CUST_INCOME_LEVEL, t0.CUST_LAST_NAME,
t0.CUST_MAIN_PHONE_NUMBER, t0.CUST_MARITAL_STATUS, t0.CUST_POSTAL_CODE, t0.CUST_SRC_ID,
t0.CUST_STATE_PROVINCE, t0.CUST_STATE_PROVINCE_ID, t0.CUST_STREET_ADDRESS, t0.CUST_TOTAL, t0.CUST_TOTAL_ID,
t0.CUST_VALID, t0.CUST_YEAR_OF_BIRTH
FROM SH.CHANNELS t3, SH.TIMES t2, SH.SALES t1, SH.CUSTOMERS t0
WHERE (((t2.CALENDAR_YEAR = ?) AND (t3.CHANNEL_DESC = ?)) AND (((t2.TIME_ID = t1.TIME_ID) AND (t3.CHANNEL_ID
= t1.CHANNEL_ID)) AND (t0.CUST_ID = t1.CUST_ID)))
INTERSECT (SELECT t4.CUST_ID, t4.COUNTRY_ID, t4.CUST_CITY, t4.CUST_CITY_ID, t4.CUST_CREDIT_LIMIT,
t4.CUST_EFF_FROM, t4.CUST_EFF_TO, t4.CUST_EMAIL, t4.CUST_FIRST_NAME, t4.CUST_GENDER, t4.CUST_INCOME_LEVEL,
t4.CUST_LAST_NAME, t4.CUST_MAIN_PHONE_NUMBER, t4.CUST_MARITAL_STATUS, t4.CUST_POSTAL_CODE, t4.CUST_SRC_ID,
t4.CUST_STATE_PROVINCE, t4.CUST_STATE_PROVINCE_ID, t4.CUST_STREET_ADDRESS, t4.CUST_TOTAL, t4.CUST_TOTAL_ID,
t4.CUST_VALID, t4.CUST_YEAR_OF_BIRTH
FROM SH.CHANNELS t7, SH.TIMES t6, SH.SALES t5, SH.CUSTOMERS t4
WHERE (((t6.CALENDAR_YEAR = ?) AND (t7.CHANNEL_DESC = ?)) AND (((t6.TIME_ID = t5.TIME_ID) AND (t7.CHANNEL_ID
= t5.CHANNEL_ID)) AND (t4.CUST_ID = t5.CUST_ID)))))
bind => [2020, Internet, 2020, Direct Sales]
```

쿼리는 1,913행을 돌려주고, 테스트는 앞의 10행만 로그에 남긴다.

로그에 남긴 결과

```
com.github.jinahya.oracle.sample.persistence.sh.Customer@783d{custId=30781,custFirstName=Abel,custLastName=V
ance,...
com.github.jinahya.oracle.sample.persistence.sh.Customer@24eb{custId=9451,custFirstName=Abraham,custLastName
=Roland,...
com.github.jinahya.oracle.sample.persistence.sh.Customer@39c7{custId=14791,custFirstName=Anand,custLastName=
Hanes,...
...
com.github.jinahya.oracle.sample.persistence.sh.Customer@cb8{custId=3256,custFirstName=Babetta,custLastName=
Salvadore,...
```

[1] Oracle. Sales History Sample Schema, README.md. https://github.com/oracle-samples/db-sample-schemas/blob/main/sales_history/README.md

[2] Oracle. Database Sample Schemas, SH Sample Schema Scripts and Objects. https://docs.oracle.com/en/database/oracle/oracle-database/23/comsc/SH-sample-schema-scripts-and-objects.html

[3] Jakarta Persistence 3.2, 2.4 Primary Keys and Entity Identity. "Every entity must have a primary key." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a132

[4] Jakarta Persistence 3.2, 3.1 Overview. "A persistence context is a set of entity instances in which for any persistent entity identity there is a unique entity instance." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#overview

[5] Oracle. VLDB and Partitioning Guide, 3.1 Partition Pruning. "In partition pruning, the optimizer analyzes FROM and WHERE clauses in SQL statements to eliminate unneeded partitions when building the partition access list." https://docs.oracle.com/en/database/oracle/oracle-database/23/vldbg/partition-pruning.html

[6] Jakarta Persistence 3.2, 11.1.9 Column Annotation. "Portable applications which make use of schema generation should

explicitly specify the precision and scale of columns of type numeric or decimal." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a14330

[7] Jakarta Persistence 3.2, 3.9 Type Conversion of Basic Attributes. "When such converted attributes are used in comparison operations with literals or parameters, the value of the literal or parameter to which they are compared must also be converted." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a2999

[8] Jakarta Persistence 3.2, 11.1.9 Column Annotation. "boolean insertable() default true; boolean updatable() default true;" https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a14330

[9] Jakarta Persistence 3.2, 2.7 Embeddable Classes. "Instances of these classes, unlike entity instances, do not have persistent identity of their own. Instead, they exist only as part of the state of the entity to which they belong." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a487

[10] Jakarta Persistence 3.2, 2.13.2 Mapped Superclasses. "Mapping information inherited from a mapped superclass can be overridden in such subclasses using the AttributeOverride and AssociationOverride annotations or corresponding XML elements." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#mapped-superclasses

[11] Jakarta Persistence 3.2, 2.4.1 Composite primary keys. "A composite primary key must either be represented and mapped as an embeddable class (see Section 11.1.17) or it must be represented as an id class and mapped to multiple fields or properties of the entity class (see Section 11.1.23)." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#composite-primary-keys

[12] Jakarta Persistence 3.2, 2.4 Primary Keys and Entity Identity. "If the composite primary key corresponds to a single field or property of the entity, the EmbeddedId annotation defined by Section 11.1.17 identifies the primary key, and the type of the annotated field or property is the primary key class." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a132

[13] Jakarta Persistence 3.2, 2.4 Primary Keys and Entity Identity. "The application must not change the value of the primary key of an entity instance after the instance is made persistent" https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a132

[14] Jakarta Persistence 3.2, 2.4.1 Composite primary keys. "The primary key class must define equals and hashCode methods." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#composite-primary-keys

[15] Jakarta Persistence 3.2, 2.4 Primary Keys and Entity Identity. "Otherwise, when the composite primary key corresponds to multiple fields or properties, the Id annotation defined by Section 11.1.22 identifies the fields and properties which comprise the composite key, and the IdClass annotation defined by Section 11.1.23 must specify the primary key class." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a132

[16] Jakarta Persistence 3.2, 2.4.1 Composite primary keys. "If the composite primary key class is represented as an id class, the names of primary key fields or properties of the primary key class and those of the entity class to which the id class is mapped must correspond and their types must be the same." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#composite-primary-keys

[17] Jakarta Persistence 3.2, 3.9 Type Conversion of Basic Attributes. "The conversion of all basic types is supported except for the following: Id attributes (including the attributes of embedded ids and derived identities), version attributes, relationship attributes, and attributes explicitly annotated as Enumerated or Temporal or designated as such in the XML descriptor. Auto-apply converters will not be applied to such attributes, and applications that apply converters to such attributes through use of the Convert annotation will not be portable." https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2.html#a2999